

Muons cosmiques

Session 2, 3

TP Césire

2018/2019

LPSC, Grenoble

by Kseniia Svirina

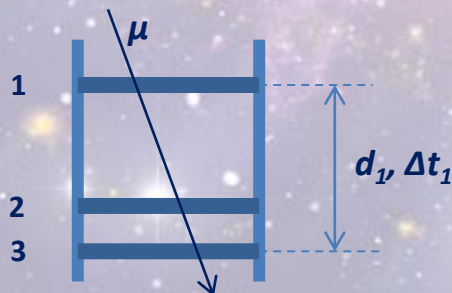
***Don't forget to save the results of the acquisition from Session 1
(save it as .txt (!) and as a picture)***

Make sure that you have the value of d_1 .

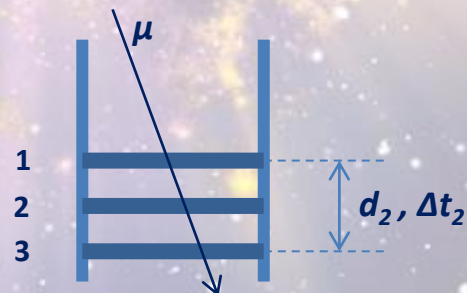
***Change the position of the upper detector, write down d_2
and start a new acquisition for 4h.***

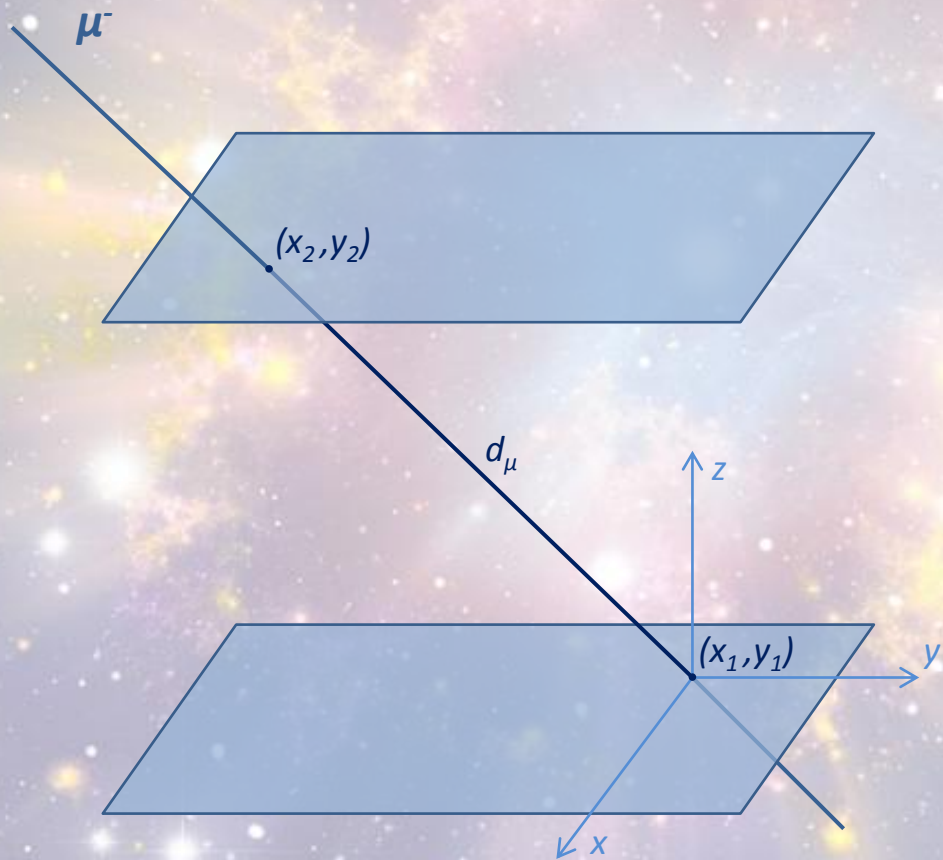
(when finished, also save it as .txt and as a picture)

Measurement 1: upper position



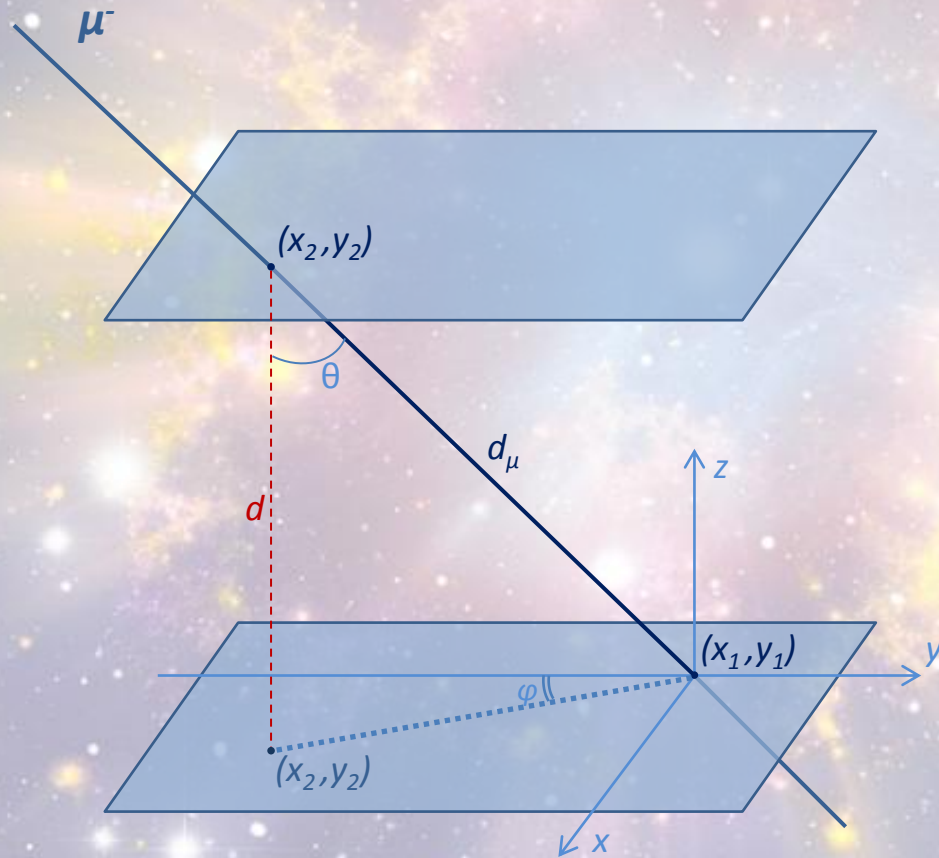
Measurement 2: lower position





$$\theta \in \left(-\frac{\pi}{2}, \frac{\pi}{2} \right)$$

$$\varphi \in [0, 2\pi)$$



$$\theta \in \left(-\frac{\pi}{2}, \frac{\pi}{2} \right)$$

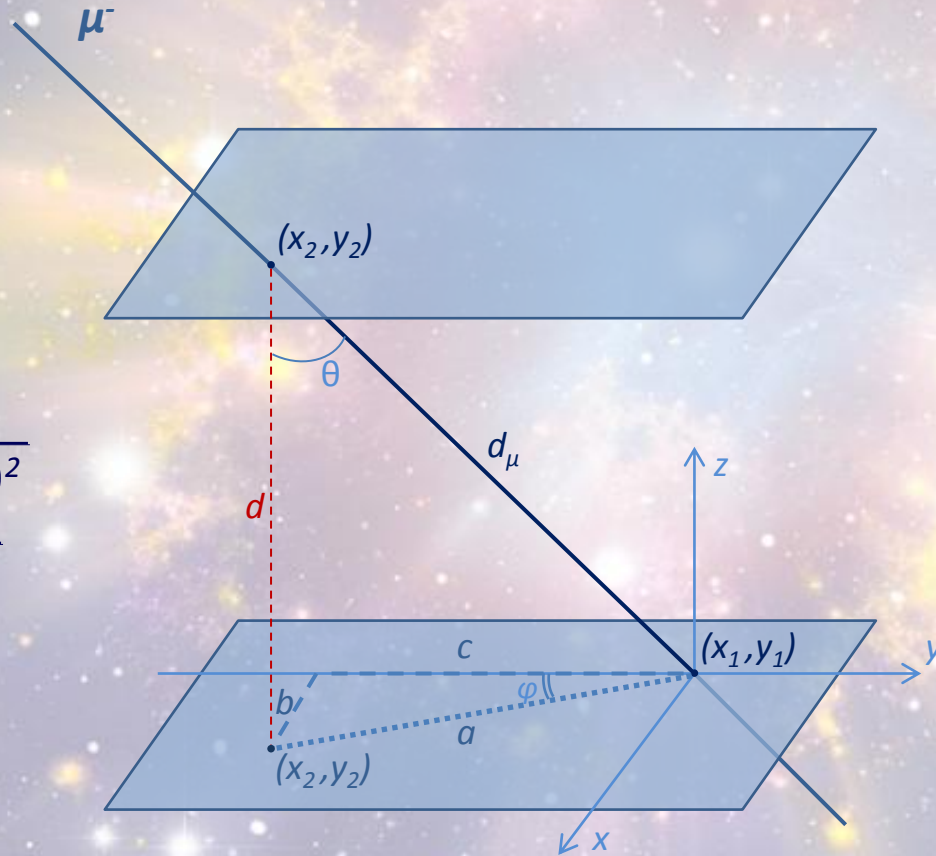
$$\varphi \in [0, 2\pi]$$

$$a = \operatorname{tg} \theta \cdot d$$

$$b = a \cdot \sin \varphi = \operatorname{tg} \theta \cdot d \cdot \sin \varphi$$

$$c = a \cdot \cos \varphi = \operatorname{tg} \theta \cdot d \cdot \cos \varphi$$

$$\begin{aligned} d_\mu &= \sqrt{d^2 + a^2} \\ &= \sqrt{d^2 + (\operatorname{tg} \theta \cdot d)^2} \\ &= \sqrt{d^2 (1 + \operatorname{tg}^2 \theta)} \end{aligned}$$



$$\theta \in \left(-\frac{\pi}{2}, \frac{\pi}{2} \right)$$

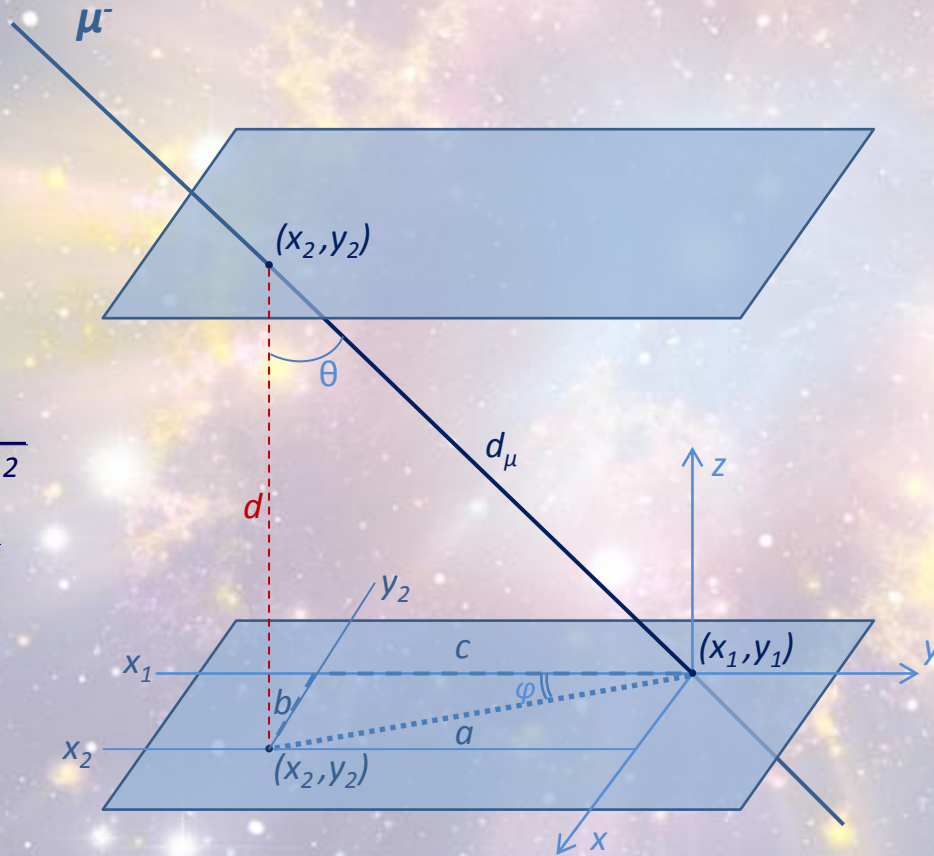
$$\varphi \in [0, 2\pi]$$

$$a = \operatorname{tg} \theta \cdot d$$

$$b = a \cdot \sin \varphi = \operatorname{tg} \theta \cdot d \cdot \sin \varphi$$

$$c = a \cdot \cos \varphi = \operatorname{tg} \theta \cdot d \cdot \cos \varphi$$

$$\begin{aligned} d_\mu &= \sqrt{d^2 + a^2} \\ &= \sqrt{d^2 + (\operatorname{tg} \theta \cdot d)^2} \\ &= \sqrt{d^2 (1 + \operatorname{tg}^2 \theta)} \end{aligned}$$



$$|x_2 - x_1| = b$$

$$|y_2 - y_1| = c$$

$$\theta \in \left(-\frac{\pi}{2}, \frac{\pi}{2} \right)$$

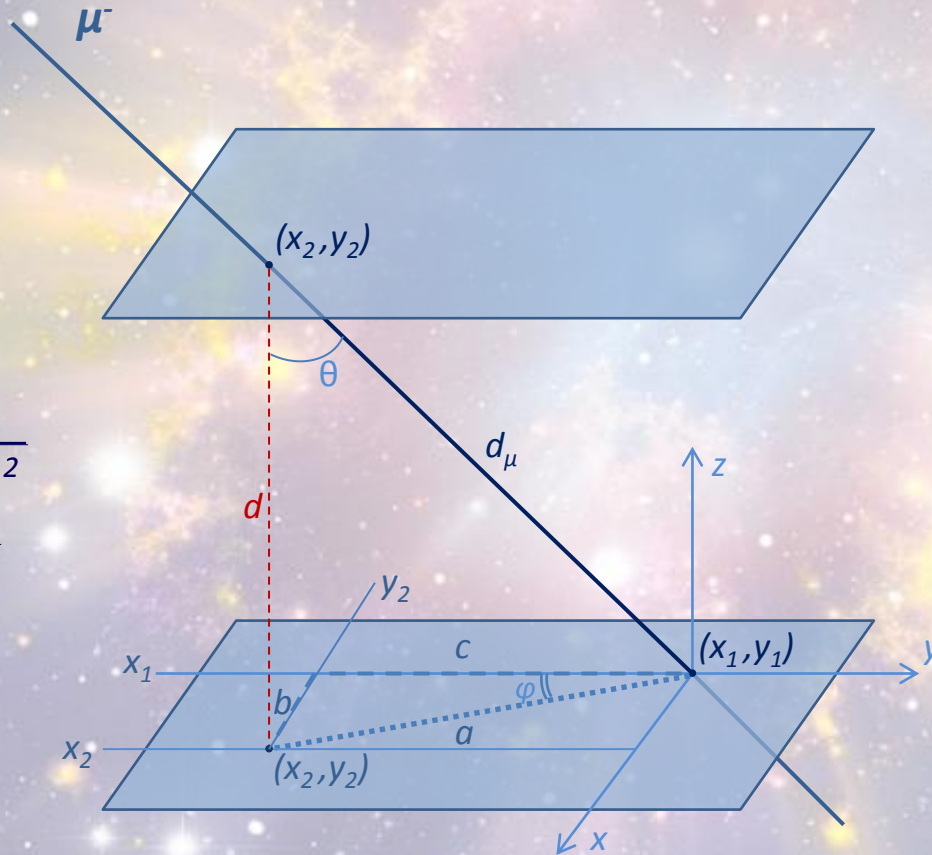
$$\varphi \in 0, 2\pi$$

$$a = \operatorname{tg} \theta \cdot d$$

$$b = a \cdot \sin \varphi = \operatorname{tg} \theta \cdot d \cdot \sin \varphi$$

$$c = a \cdot \cos \varphi = \operatorname{tg} \theta \cdot d \cdot \cos \varphi$$

$$\begin{aligned} d_\mu &= \sqrt{d^2 + a^2} \\ &= \sqrt{d^2 + (\operatorname{tg} \theta \cdot d)^2} \\ &= \sqrt{d^2 (1 + \operatorname{tg}^2 \theta)} \end{aligned}$$



$$|x_2 - x_1| = b$$

$$|y_2 - y_1| = c$$

$$x_2 = x_1 + b = x_1 + \operatorname{tg} \theta \cdot d \cdot \sin \varphi$$

$$y_2 = y_1 + c = y_1 + \operatorname{tg} \theta \cdot d \cdot \cos \varphi$$

We want to know the mean distance that a muon passes in the volume between the 1st and the 3rd detectors.

$$d_{\mu}^{\text{mean}} = \frac{\sum_{i \in [1, N]} d_{\mu}^i}{N}$$

Generate randomly $(x_1, y_1, \theta, \varphi)$ with respect that

$$\begin{cases} 0 < x_1 < x_{\max} \\ 0 < y_1 < y_{\max} \\ -\frac{\pi}{2} < \theta < \frac{\pi}{2} \\ 0 < \varphi < 2\pi \end{cases}$$

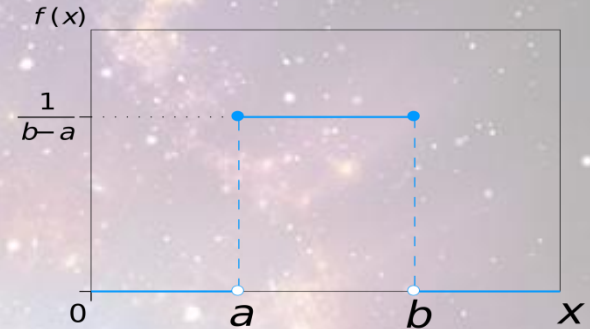
For each set of $(x_1, y_1, \theta, \varphi)$ calculate

$$\begin{cases} d_{\mu} = \sqrt{d^2 \cdot 1 + \tan^2 \theta} \\ x_2 = x_1 + \tan \theta \cdot d \cdot \sin \varphi \\ y_2 = y_1 + \tan \theta \cdot d \cdot \cos \varphi \end{cases} \quad \text{check that} \quad \begin{cases} 0 < x_2 < x_{\max} \\ 0 < y_2 < y_{\max} \end{cases}$$

if YES => save this d_{μ}

Distributions for x_1 , y_1 and φ are uniform

```
for i in range(0,10000):  
    x1=uniform(0,xmax)  
    y1=uniform(0,ymax)  
    phi=uniform(0,2*np.pi)  
    ...  
    x2=x1+a*np.sin(phi)  
    y2=y1+a*np.cos(phi)  
  
    if 0<x2<xmax and 0<y2<ymax:  
        dmua = ...
```

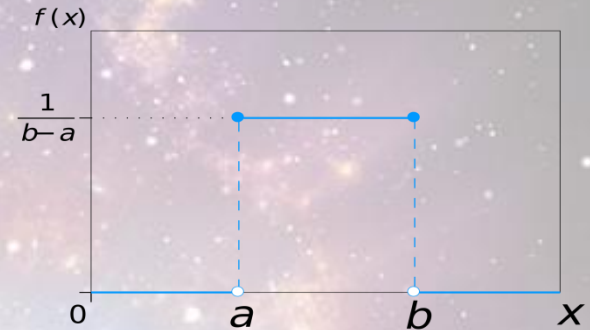


Distribution for θ is not uniform.

According to the physics of our experiment, the muons pass mostly vertically, i.e. θ 's have a higher probability to be close to 0 than to be close to $\pi/2$ (horizontal trajectory).

Distributions for x_1 , y_1 and φ are uniform

```
for i in range(0,10000):  
    x1=uniform(0,xmax)  
    y1=uniform(0,ymax)  
    phi=uniform(0,2*np.pi)  
    ...  
    x2=x1+a*np.sin(phi)  
    y2=y1+a*np.cos(phi)  
  
    if 0<x2<xmax and 0<y2<ymax:  
        dmdu = ...
```

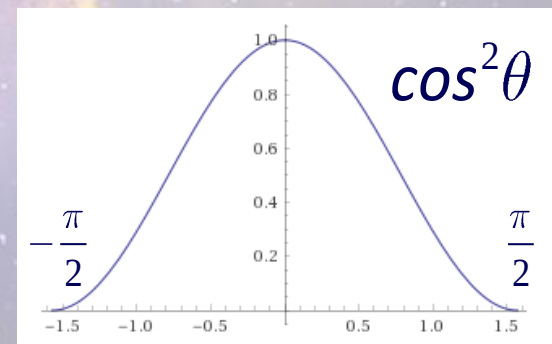


Distribution for θ is not uniform.

According to the physics of our experiment, the muons pass mostly vertically, i.e. θ 's have a higher probability to be close to 0 than to be close to $\pi/2$ (horizontal trajectory).

The more appropriate distribution is $\cos^2\theta$

for
$$-\frac{\pi}{2} < \theta < \frac{\pi}{2}$$



d-mean simulation:

```
import random as rd  
import numpy as np  
import matplotlib.pyplot as plt
```

where

*randint(start, end) – random integer from the
given interval*

uniform(x, y) – uniform distribution

*NumPy - is a package for scientific computing with
Python*

Math – for mathematical functions

matplotlib.pylab – for plots

plt.hist(...)

plt.show()

d-mean simulation:

```
import random as rd
import numpy as np
import matplotlib.pyplot as plt
```

where

randint(start, end) – random integer from the given interval

uniform(x, y) – uniform distribution

NumPy - is a package for scientific computing with Python

Math – for mathematical functions

matplotlib.pylab – for plots

plt.hist(...)

plt.show()

```
def f(x):
    return ... #Create the func. (cosθ)**2
l=[] #List for the results dmu
N=100000 #Number of the events
For k in range(0,N)
    x1=rd.uniform(0,xmax)
    y1=...
    phi=...
#Creation of the variable theta
a=rd.uniform(-np.pi/2,np.pi/2)
b=rd.uniform(0,1)
if f(a)>=y:
    theta=a
x2=...
y2=...
if 0<x2<xmax and 0<y2<ymax
    dmu=...
    l.append(dmu)
...
```

d-mean simulation:

```
import random as rd
import numpy as np
import matplotlib.pyplot as plt
```

where

randint(start, end) – random integer from the given interval

uniform(x, y) – uniform distribution

NumPy - is a package for scientific computing with Python

Math – for mathematical functions

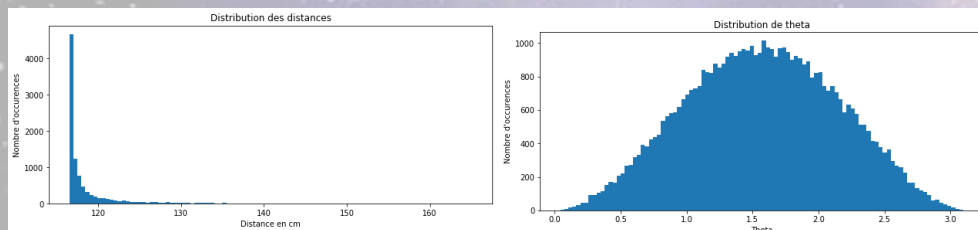
matplotlib.pylab – for plots

plt.hist(...)

plt.show()

```
def f(x):
    return ... #Create the func. (cosθ)**2
l=[] #List for the results dmu
N=100000 #Number of the events
For k in range(0,N)
    x1=rd.uniform(0,xmax)
    y1=...
    phi=...
#Creation of the variable theta
a=rd.uniform(-np.pi/2,np.pi/2)
b=rd.uniform(0,1)
if f(a)>=y:
    theta=a
x2=...
y2=...
if 0<x2<xmax and 0<y2<ymin
    dmu=...
    l.append(dmu)
...
```

Please build the distributions: $N^{\text{events}}(\text{dmu})$, $N^{\text{events}}(\text{theta})$



Session 3: Gaussian fit example

```
import matplotlib.pyplot as plt
import numpy as np
from math import *
from scipy.optimize import curve_fit

d1=... #Use results of the simulation
d2=...

def file_result() :
    fichier = open("spectre1.txt", "r")
    fichier1 = open("spectre2.txt", "r")
    val=fichier.readlines()
    val0=fichier1.readlines()
    fichier.close()
    fichier1.close()
    val1=[val[i].split("\n") for i in
          range(20000)]
    val10=[val0[i].split("\n") for i in
           range(20000)]
    val2=[int(val1[i][0]) for i in range(20000)]
    val20=[int(val10[i][0]) for i in
           range(20000)]

    return(val2,val20)

def gaussienne(x,a,b,c) :
    return(a*np.exp(-(x-b)**2/c))

X=np.linspace(0,len(file_result()[0]),
              len(file_result()[0]))

Y=file_result()[0]
Y1=file_result()[1]

T=curve_fit(gaussienne,X,Y,[100,8500,20006])
T1=curve_fit(gaussienne,X,Y1,[9,7500,10006])
a=T[0][0]
b=T[0][1]
c=T[0][2]
...
```

```
#Use the TAC calibration
def canal_time(c):
    t= ... *c - ...
    return(t)
t1=canal_time(b)
t2=canal_time(b1)
...
print b
print b1

Z=[gaussienne(i,a,b,c) for i in range(len(X))]
Z1=[gaussienne(i,a1,b1,c1) for i in range(len(X))]

plt.figure('Shortest distance between detectors')
plt.title('Shortest distance between detectors')
plt.xlabel('Channel')
plt.ylabel('Numbers of events')
plt.grid()
plt.plot(X,Y,'#59a85f',lw=1)
plt.plot(X,Z,'r',lw=3)
#plt.axis([12000,20000,0,100])

plt.figure('Longest distance between detectors')
plt.title('Longest distance between detectors')
plt.xlabel('Channel')
plt.ylabel('Numbers of impacts')
plt.grid()
plt.plot(X,Y1,'#59a85f',lw=1)
plt.plot(X,Z1,'r',lw=3)
plt.show()

def vitesse(d1,d2,t1,t2) :
    v=(d2-d1)/((t1-t2)*10**-9)
    v_rel=v/299792458*100
    return(v," that is ",v_rel,"% of c")
print(vitesse(d1,d2,t1,t2))
```


Session 3: Gaussian fit example

```
import matplotlib.pyplot as plt
import numpy as np
from math import *
from scipy.optimize import curve_fit

d1=... #Use results of the simulation
d2=...

def file_result() :
    fichier = open("spectre1.txt", "r")
    fichier1 = open("spectre2.txt", "r")
    val=fichier.readlines()
    val0=fichier1.readlines()
    fichier.close()
    fichier1.close()
    val1=[val[i].split("\n") for i in
          range(20000)]
    val10=[val0[i].split("\n") for i in
           range(20000)]
    val2=[int(val1[i][0]) for i in range(20000)]
    val20=[int(val10[i][0]) for i in
           range(20000)]

    return(val2,val20)

def gaussienne(x,a,b,c) :
    return(a*np.exp(-(x-b)**2/c))

X=np.linspace(0,len(file_result()[0]),
              len(file_result()[0]))
Y=file_result()[0]
Y1=file_result()[1]

T=curve_fit(gaussienne,X,Y,[100,8500,20006])
T1=curve_fit(gaussienne,X,Y1,[9,7500,10006])
a=T[0][0]
b=T[0][1]
c=T[0][2]
...
```

```
#Use the TAC calibration
def canal_time(c):
    t= ... *c - ...
    return(t)
t1=canal_time(b)
t2=canal_time(b1)
...
print b
print b1

Z=[gaussienne(i,a,b,c) for i in range(len(X))]
Z1=[gaussienne(i,a1,b1,c1) for i in range(len(X))]

plt.figure('Shortest distance between detectors')
plt.title('Shortest distance between detectors')
plt.xlabel('Channel')
plt.ylabel('Numbers of events')
plt.grid()
plt.plot(X,Y,'#59a85f',lw=1)
plt.plot(X,Z,'r',lw=3)
#plt.a

plt.fi
plt.ti
plt.xl
plt.yl
plt.gr
plt.pl
plt.pl
plt.sh

def vi
v=
v_
re
print(
```

